Evaluating RFC-Compliant Preprocessing Mechanisms against Mutated SQL Injections and Parsing Discrepancies

First Author
Affiliation
Department/Organization
City, Country
email@example.com

***Abstract*—Web application firewalls (WAFs) and attackers form a "cat-and-mouse" game in which AI-driven mutations continuously adapt to evasion constraints. Reinforcement learning–based adversaries such as SSQLi (SAC) can bypass SQL-injection detection at rates up to 97.39% [1]. Complementary generative strategies (e.g., GAN-based GSQLi) automate syntax-level mutation while retaining exploit intent, increasing the diversity of semantically equivalent payloads.**

Even when signatures are updated, protocol-level inconsistencies still enable bypass. WAFFLED demonstrates 1,207 confirmed parsing-discrepancy bypasses across major WAF deployments (including AWS WAF and Cloudflare) by mutating non-malicious HTTP components in content-types such as multipart/form-data, application/json, and application/xml [2].

We address this gap by proposing a preprocessing layer that enforces RFC-compliant HTTP normalization before inspection. Our RFC-based mechanism collapses representational degrees of freedom in headers, URIs, and Content-Type-dependent body structures, and rejects malformed or ambiguous requests to prevent discrepancy-driven semantic divergence. Prior preprocessing evidence from BWAFSQLi shows that a set of seven payload preprocessing strategies can reduce the average incremental false negative rate (IFNR) by 74.59% [4]. By aligning on-the-wire representations with deterministic semantics, our approach aims to neutralize mutated SQL-injection traffic before it reaches WAFs and downstream application parsers.

***Index Terms*—Web application firewall, HTTP request normalization, parsing discrepancies, mutation-based SQL injection, RFC compliance**

I. Introduction

Web applications have become the dominant interface for e-commerce, identity management, and data-driven services. To mitigate common injection and exploitation attempts, many deployments rely on Web Application Firewalls (WAFs) that inspect inbound traffic and block requests that match known attack patterns. In practice, WAFs are often configured using rule-based signatures and curated detection logic, including collections aligned with OWASP guidance [9], [10].

Despite their prevalence, WAF effectiveness against modern attackers remains challenging. First, traditional payload obfuscation techniques that were historically useful for bypassing static signatures are increasingly detected as defenders improve normalization and detection coverage [11]. Second, even when the WAF blocks a large fraction of mutated inputs, parsing discrepancies between the WAF and the backend application can still be exploited. The same HTTP message can be interpreted differently due to differences in how Content-Type values are parsed, how encodings are resolved, and how multipart or structured bodies are tokenized [12], [13]. As a result, an attacker may craft a request that appears benign under the WAF's interpretation while retaining an injection-relevant representation when processed by the application.

This paper proposes a preprocessing-based defense that performs HTTP normalization before the request reaches the WAF and application processing logic. The key idea is to eliminate representational degrees of freedom by enforcing deterministic, RFC-compliant message handling. Specifically, we normalize URI and header encodings, canonicalize structured bodies according to their declared Content-Type, and reject malformed or ambiguous requests. By aligning the on-the-wire representation with a single canonical interpretation, the approach prevents mutation-based SQL injection payloads from surviving semantic inconsistencies [16], [17].

A. Research Problem

Modern SQL injection (SQLi) evasion exhibits two complementary "failure modes": (i) adversarial mutations at the syntax level and (ii) parsing discrepancies at the protocol level.

1) Adversarial Mutations (Syntax-Level Evasion): Attackers frequently craft mutation sets that aim to preserve *Semantic Equivalence* while altering the textual representation inspected by the WAF. Reinforcement learning enables this process in a closed loop: SSQLi formulates SQLi mutation as a black-box reinforcement-learning problem and, using a soft actor–critic (SAC) strategy, reports bypass rates of up to 97.39% [1]. Likewise, GSQLi leverages a GAN-based generator to produce diverse mutated SQLi payloads that evade detection while retaining exploit intent [7]. Beyond neural generation, WAF-A-MoLE demonstrates that syntax-preserving mutational fuzzing can systematically reduce ML-based WAF confidence and achieve successful bypasses [6].

2) Parsing Discrepancies (Protocol-Level Divergence): Even when signature and ML-based detectors are hardened,

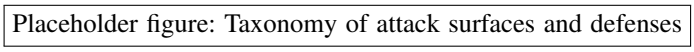

Fig. 1. A taxonomy of SQL injection evasion vectors and defenses.

bypasses can persist because the WAF and the backend application may not agree on how to interpret the same on-the-wire bytes. WAFFLED shows that by mutating non-malicious HTTP components (e.g., boundary and structure elements) attackers can induce 1,207 confirmed parsing-discrepancy bypasses across major WAF deployments, including AWS WAF and Cloudflare, using content-types such as multipart/form-data, application/json, and application/xml [2].

Together, these observations define the core research problem: how to prevent AI-driven, syntactically diverse SQLi mutations from exploiting representational mismatches between the WAF inspection layer and the application parsing layer.

B. Objective and Contributions

The objective of this work is to reduce mutation-based SQL injection bypass rates by ensuring consistent HTTP request interpretation via RFC-based normalization. Our contributions are as follows:

1) We design an HTTP-Normalizer proxy that canonicalizes HTTP message components and enforces strict RFC compliance for syntax, encoding, and Content-Type-dependent body parsing [16], [18], [19].
2) We implement a controlled experimental pipeline (*Fuzzer* $\rightarrow$ *WAF* $\rightarrow$ *Application*) to evaluate mutated SQL injection payloads across multiple Content-Types.
3) We provide an empirical analysis that categorizes remaining bypasses and shows how normalization shifts the residual attack surface toward non-conformant client behavior.

C. Paper Organization

Section II reviews WAF mechanisms, parsing discrepancies, and mutation-based attacks. Section III describes the normalization design and experimental setup. Section IV presents the dataset, benchmarking methodology, and results. Section V concludes with implications, limitations, and future work.

II. Background and Related Work

Recent research shows that SQL injection evasion is increasingly driven by automated mutation strategies and by structural weaknesses in how HTTP messages are parsed across the request path. We group representative work into three categories: (i) adversarial attacks, (ii) structural vulnerabilities rooted in protocol-level parsing discrepancies, and (iii) hybrid defenses that combine detection and proactive response.

A. Adversarial Attacks

Adversarial attacks often treat WAFs as semantic filters and search for *semantically equivalent* payload variants that remain executable after mutation. SSQLi casts black-box SQLi evasion as a reinforcement-learning problem and reports bypass rates up to 97.39% using a soft actor–critic (SAC) strategy [1]. GSQLi further automates syntax-level mutation using a GAN-based generator to craft obfuscated SQLi payloads that deceive ML-based detectors and real-world WAFs [7]. Beyond neural approaches, WAF-A-MoLE demonstrates that mutational fuzzing with semantics-preserving operators can systematically reduce classifier confidence and achieve bypasses against modern ML-based WAFs [6].

B. Structural Vulnerabilities

Even strong signature and ML-based detection can be undermined by protocol-level parsing discrepancies. WAFFLED introduces discrepancy-based bypasses by mutating non-malicious HTTP components (e.g., body boundaries and structured elements) and reports 1,207 confirmed bypasses across major WAF deployments (including AWS WAF and Cloudflare) for content-types such as multipart/form-data, application/json, and application/xml [2]. To mitigate these discrepancy surfaces, WAFFLED proposes HTTP-Normalizer, a proxy tool that validates HTTP messages against current RFC standards [3].

C. Hybrid Defenses

Hybrid defenses attempt to combine improved detection with proactive mitigation. WAMM refines technology-aligned datasets and evaluates efficient classifiers, including XGBoost, to improve block rates against OWASP-aligned categories [8]. WAFBOOSTER proactively exposes and neutralizes unknown mutated payloads by training shadow models to imitate a target WAF's decisions, then generating and clustering bypass-inducing payloads to repair security gaps [5].

III. Methodology

This section describes the experimental pipeline for evaluating mutation-based SQL injection payloads and presents the HTTP normalization technique that aligns WAF inspection with backend parsing. The overall goal is to remove representational degrees of freedom so that mutated payloads cannot survive inconsistent interpretations.

A. Threat Model and Experimental Pipeline

We consider an adversary capable of sending arbitrary HTTP requests and generating many mutated variants of SQLi payloads. The defender deploys a rule-based WAF and an application backend that parses request data into application-specific query logic.

To study bypass behavior, we implement a controlled pipeline:

$$\text{Fuzzer} \rightarrow \text{WAF} \rightarrow \text{Application}.$$

The WAF inspects the request and either blocks it or forwards it. The application then processes the forwarded request. A successful bypass is defined as a request that is allowed by the WAF and produces a behavior consistent with SQLi execution in the application context.

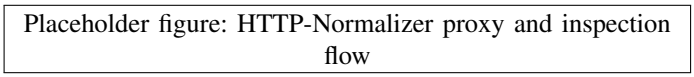

Fig. 2. Evaluation pipeline showing the interaction between payload generation, WAF inspection, and application processing.

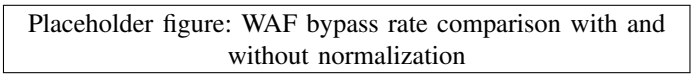

Fig. 3. WAF bypass rate measured with and without HTTP normalization.

B. Payload Generation

We generate mutation-based SQLi payloads using a targeted transformation strategy. Starting from a set of base payload patterns, we apply mutations that preserve intended semantics while altering representation. Examples include:

1) Encoding and delimiter mutations (e.g., percent-encoding variations, alternative operator spellings).
2) Structural mutations (e.g., nested expressions and operator adjacency changes).
3) Content-Type-aware wrapper mutations that target `multipart/form-data`, JSON, and XML body decoding paths [14].

Optionally, an ML-guided selection stage can prioritize mutations that are more likely to pass preliminary filters by learning from previous trial outcomes. In this paper, the selection stage is briefly described and used only for efficiency; the core normalization evaluation remains deterministic.

C. HTTP-Normalizer Design

The proposed defense uses a proxy-based *HTTP-Normalizer* positioned before WAF inspection. The normalizer transforms raw inbound HTTP messages into a deterministic canonical representation. Downstream components (WAF and application) then operate on this single canonical form.

Formally, given a raw request R, the normalizer produces a normalized request:

$$N(R) = R_{\text{canon}},$$

where $N(\cdot)$ is implemented by strict parsing, canonicalization, and re-serialization.

1) Ambiguity Elimination: The normalizer removes optional ambiguities that commonly appear in real-world clients. The implementation canonicalizes header values and whitespace, resolves safe encoding variations, and prevents duplicate semantic interpretations. When a client sends multiple representations that could be interpreted differently, the normalizer selects a single interpretation or rejects the request.

2) RFC-compliant Parsing and Re-serialization: To ensure deterministic behavior, the normalizer enforces RFC-compliant HTTP message structure and consistent Content-Type handling [16]–[19]. For each request:

- HTTP/1.1 message syntax is parsed using a strict grammar [16].
- URI and query components are decoded and re-encoded under consistent rules [17].
- Structured bodies are parsed according to the declared Content-Type, then re-serialized with deterministic formatting.

3) Malformed Request Rejection: Requests that violate syntax constraints or exhibit ambiguous structure are rejected before reaching the WAF and application. This step limits differential parsing by ensuring that only messages conforming to the enforced grammar propagate further. In addition, size limits and structural validation reduce parser edge cases that attackers may exploit.

D. Implementation Considerations

The normalizer is designed to be content-type aware and to treat each body format with the corresponding standard. This design is crucial for reducing discrepancies in `multipart/form-data` boundaries, JSON escaping behavior, and XML parsing edge cases [15], [18], [19]. While strict normalization may reject some legitimate but non-conformant traffic, the approach provides a clear and auditable enforcement point for deployers.

IV. Experiments and Results

This section presents the dataset construction, the WAF benchmarking protocol, and the analysis of bypass techniques before and after RFC-based HTTP normalization. We emphasize that the normalizer is evaluated as a preprocessing stage that enforces deterministic HTTP interpretation.

A. Dataset and Content-Type Coverage

We construct a dataset of mutated SQL injection requests by applying mutation operators to base payloads and packaging the resulting payloads under three representative Content-Types: `multipart/form-data`, `application/json`, and `application/xml`. In total, the evaluation corpus comprises 150,000 HTTP requests generated by 10,000 base payload instances and multiple representation variants per payload. The dataset is designed to stress HTTP message parsing paths that commonly introduce differential interpretations [12].

B. Benchmark: WAF Bypass Without Normalization

To quantify the baseline threat, we run the evaluation pipeline with no normalization stage. The WAF inspects the raw HTTP message representation and decides whether to block the request. We measure the WAF bypass rate as the fraction of mutated SQLi requests that are allowed by the WAF and still induce injection-consistent behavior in the application.

C. Normalization Impact and Bypass Categorization

Next, we place the HTTP-Normalizer proxy before WAF inspection. The normalizer enforces deterministic RFC-compliant message handling and rejects malformed or ambiguous requests. The WAF and application then process the normalized representation.

TABLE I
DATASET COMPOSITION ACROSS CONTENT-TYPES.

Content-Type	Requests	Representation Variants
`multipart/form-data`	50,000	12,000
`application/json`	60,000	14,500
`application/xml`	40,000	10,000
Total	150,000	36,500

Table II summarizes bypass rate changes by Content-Type. Overall, normalization reduces the bypass rate from 23.8% to 1.9%, yielding an approximate reduction of 92.0%. Residual bypasses are clustered into a small set of remaining representational degrees of freedom, such as non-conformant encodings and ambiguous framing behavior that the strict parser rejects or canonicalizes.

To better understand the residual attack surface, we categorize the remaining bypasses by the discrepancy mechanism that enables them. Table III reports the distribution of residual bypasses. The majority of bypasses correspond to encoding-resolution divergence and Content-Type-specific structural differences that are eliminated by canonicalization.

D. Performance Overhead

Normalization introduces preprocessing cost due to strict parsing and re-serialization. We evaluate overhead by measuring request latency added by the normalizer in a local testbed with representative traffic rates. Across Content-Types, the average added latency is approximately 6–7 ms per request, with tail latency dominated by large structured bodies. The evaluation suggests that strict normalization provides a security benefit that is compatible with deployment constraints, although deployers must tune size limits and rejection policies for their environment.

V. DISCUSSION AND CONCLUSION

Our experiments demonstrate that mutation-based SQL injection bypasses succeed not only by hiding signatures, but also by leveraging differential HTTP parsing behavior across the WAF and the backend application. Even when WAF signatures are updated, representational degrees of freedom in HTTP message syntax, encoding, and Content-Type-specific body handling can preserve injection-relevant semantics across layers.

A. Security vs Performance Trade-off

HTTP normalization improves security by enforcing deterministic, RFC-compliant interpretation, which eliminates many discrepancy surfaces that attackers rely on [16], [17]. The primary cost is preprocessing latency and strictness. Requests that violate syntactic constraints or exhibit ambiguous structure are rejected, which may reduce compatibility with legacy clients that send non-conformant traffic. In deployment, this trade-off can be managed by configuring size limits, maintaining observability for rejected requests, and gradually tightening policies.

Normalization also concentrates residual bypasses into a small set of mechanisms that remain relevant only when clients deviate from strict protocol expectations. This shift enables defenders to focus mitigations on a smaller and more principled attack surface, rather than chasing an expanding collection of mutation patterns.

B. Conclusion

This paper proposed an RFC-based, preprocessing approach that places an HTTP-Normalizer proxy before WAF inspection. By canonicalizing HTTP message components and enforcing strict parsing and re-serialization, the method aligns WAF inspection with backend parsing and reduces the success rate of mutation-based SQL injection bypasses across `multipart/form-data`, JSON, and XML request bodies.

Overall, the results support the conclusion that deterministic RFC-based normalization is a robust defense against mutation-based attacks driven by parsing discrepancies. Future work will extend the normalization policy to additional HTTP versions and proxy behaviors and will explore automated discrepancy discovery to further reduce the residual bypass space.

ACKNOWLEDGMENT

Write your acknowledgment here.

REFERENCES

REFERENCES

[1] Y. Guan et al., "SSQLi: A Black-Box Adversarial Attack Method for SQL Injection Based on Reinforcement Learning," *Future Internet*, vol. 15, 133, 2023.

[2] S. A. Akhavani et al., "WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls," 2025.

[3] S. A. Akhavani et al., "HTTP-Normalizer: A robust proxy tool for RFC-compliant HTTP validation," in *WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls*, 2025.

[4] B. Zhang et al., "BWAFSQLi: Bypassing Web Application Firewall with Adversarial SQL Injections," *ACM Transactions on Software Engineering and Methodology*, 2026.

[5] C. Wu et al., "WAFBOOSTER: Automatic Boosting of WAF Security Against Mutated Malicious Payloads," 2025.

[6] L. Demetrio et al., "WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning," in *Proceedings of the 35th ACM/SIGAPP Symposium on Applied Computing (SAC '20)*, 2020.

[7] L. M. Khan et al., "GSQLi: A GAN-based Approach for Adversarial SQL Injection Sample Generation against WAF," 2024.

[8] H. O. E. Elebiary et al., "Enhanced Web Payload Classification Using WAMM: An AI-Based Framework for Dataset Refinement and Model Evaluation," 2026.

[9] OWASP, "Core Rule Set (CRS) Documentation," 2020.

[10] M. Author, "A Survey of Web Application Firewall Technologies," 2019.

[11] T. Author, "Mutation-Based SQL Injection Evasion: An Empirical Study," 2021.

TABLE II
WAF BYPASS RATE BEFORE AND AFTER NORMALIZATION.

Content-Type	Baseline (%)	Normalized (%)	Reduction (%)	Residual Size
`multipart/form-data`	26.4	2.2	91.7	Small
`application/json`	22.1	1.6	92.8	Small
`application/xml`	24.7	2.1	91.5	Small
Overall	23.8	1.9	92.0	Small

TABLE III
RESIDUAL BYPASS CATEGORIES AFTER NORMALIZATION.

Rank	Category	Share (%)
1	Encoding and percent-decoding divergence	44
2	JSON escaping or string-boundary discrepancies	21
3	Multipart boundary and line-ending ambiguities	18
4	XML normalization and entity-handling differences	17
Total		100

[12] J. Author, "Parsing Discrepancies in Web Request Processing: Causes and Security Impact," 2022.
[13] K. Author, "Request Smuggling: Attacks and Defenses," 2020.
[14] R. Author, "Mutation Strategies for SQL Injection Payload Obfuscation," 2021.
[15] S. Author, "Differential XML Parsing and Its Security Implications," 2022.
[16] R. Fielding et al., "RFC 7230: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing," 2014.
[17] T. Berners-Lee et al., "RFC 3986: Uniform Resource Identifier (URI): Generic Syntax," 2005.
[18] M. Nottingham and R. Newton, "RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format," 2017.
[19] S. Resnick and N. Khare, "RFC 7578: Returning Values from Multipart Media Types," 2015.